

TESTE 1

1. Um sistema operativo é um programa que:
 - a. corre permanentemente num computador e gere os recursos de hardware para os utilizadores.
 - b. verifica periodicamente o estado do hardware e gera avisos para a substituição de componentes.
 - c. permite o arranque do computador (“bootstrap”) e depois transfere o controlo para o utilizador.
2. Qual a relação entre os sistemas operativos iOS e Android com o UNIX?
 - a. não existe nenhuma relação entre os dois sistemas operativos e o UNIX.
 - b. o iOS descende do BSD UNIX via MACOS X, o Android foi desenhado de raiz.
 - c. ambos descendem de sistemas UNIX, o BSD UNIX e o Linux, respectivamente.
3. Um sistema operativo que executa, num único processador, vários processos longos e computacionalmente intensivos sem interrupção:
 - a. é bom para “multiprogramming” mas não para “multitasking”.
 - b. é bom para “multitasking” mas não para “multiprogramming”.
 - c. não é bom para “multitasking” nem para “multiprogramming”.
4. O “Basic Input/Output System” (BIOS) de um computador é:
 - a. a parte do sistema operativo que lida com toda a interacção com os dispositivos de I/O.
 - b. a componente do sistema operativo que lida apenas com dispositivos de I/O mais simples.
 - c. um programa que carrega o “kernel” do sistema operativo para memória no arranque do computador.
5. A função do programa “loader” do sistema operativo é:
 - a. gerar código binário executável para uma arquitectura de microprocessador a partir de código em linguagem C.
 - b. transferir um binário executável para memória transformando-o num processo com espaço de endereçamento e PCB.
 - c. identificar funções sem código binário associado, localizá-las em bibliotecas e gerar um binário executável.
6. O “Process Control Block” (PCB) é:
 - a. uma estrutura de dados onde é mantida informação sobre o estado de execução de um processo.
 - b. o bloco de código do sistema operativo responsável pelo controlo da execução dos processos.
 - c. para além da fila “ready”, a outra fila de processos que esperam por tempo de microprocessador.

7. Num “context switch”, o sistema operativo:

- a. troca o processo corrente pelo processo seguinte sem guardar ou carregar qualquer tipo de informação contida nos registos ou PCBs.
- b. guarda os valores dos registos no PCB do processo que sai e carrega os registos com os valores guardados no PCB do processo que entra.
- c. guarda os valores dos registos no PCB do processo que entra e carrega os registos com os valores guardados no PCB do processo que sai.

8. Quantos identificadores de processos são imprimidos pelo programa que se segue?

```
/* includes & defines */  
  
int main() {  
    fork();  
    fork();  
    fork();  
    fork();  
    fork();  
    printf("%d\n", getpid());  
    return 0;  
}
```

- a. 32.
- b. 5.
- c. 6.

9. Se um processo P termina sem esperar pelo término de um processo filho Q:

- a. Q não sofre qualquer alteração no seu estado e continua a executar.
- b. Q torna-se um processo “zombie”, sendo eliminado pelo processo “init”.
- c. Q torna-se um processo orfão, sendo adoptado pelo processo “init”.

10. Uma “Application’s Programmer Interface” (API) é:

- a. uma interface gráfica oferecida por alguns sistemas operativos que proporciona aos programadores um ambiente mais amigável de desenvolvimento.
- b. um conjunto de exemplos de aplicações fornecidas com o sistema operativo e que facilitam a escrita de novas aplicações pelos programadores.
- c. uma especificação de um conjunto de funções disponíveis para o programador, incluindo os nomes, tipos dos parâmetros e tipos dos valores de retorno.

11. As “pipes” são um exemplo de mecanismo de comunicação entre processos suportados pelo “kernel” do UNIX. A troca de informação é:

- a. bidirecional, entre dois quaisquer processos.
- b. unidirecional, entre dois quaisquer processos.
- c. unidirecional, entre processos pai-filho.

12. O seguinte programa:

```

/* includes & defines */

int main(int argc, char* argv[]) {
    char* args[8];
    int i;
    for(i = 0; i < argc - 1; i++)
        args[i] = argv[i+1];
    args[i] = NULL;
    execvp(args[0], args);
    return 0;
}

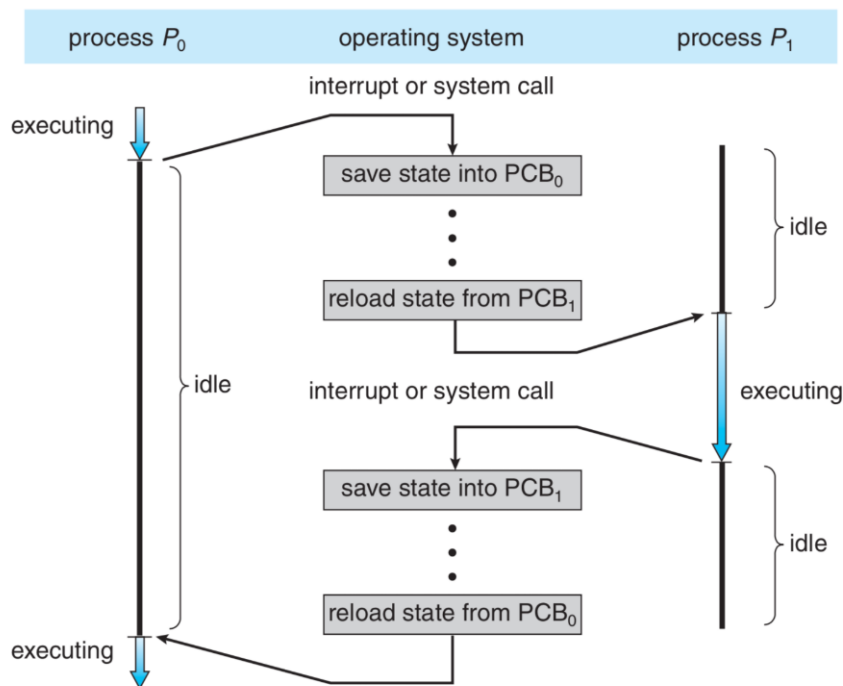
```

- executa o programa dado em `argv[1]` com os argumentos em `argv[2]`, etc.
- executa o programa dado em `argv[0]` com os argumentos em `argv[1]`, etc.
- falha sempre na execução da chamada ao sistema “`execvp`” e retorna 0.

13. Tipicamente, um processo executa:

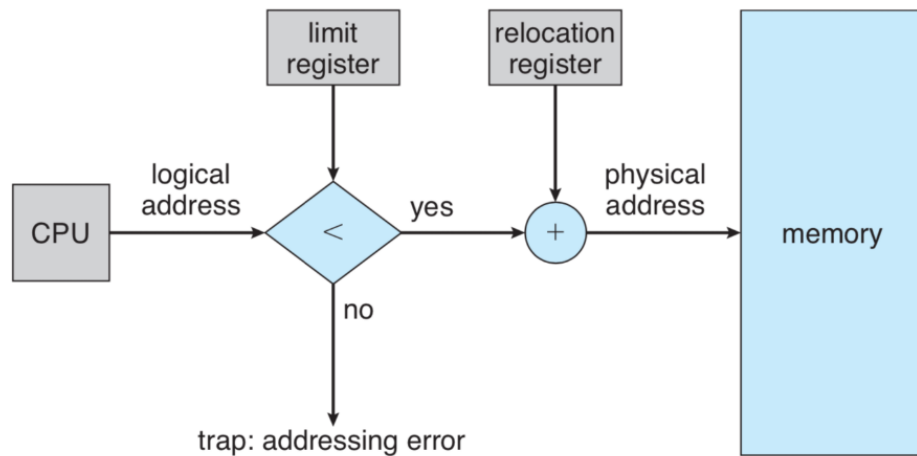
- alternando entre “bursts” de microprocessador e de I/O.
- com um “burst” de microprocessador, o I/O é todo no fim.
- todo o I/O no início, seguido de “burst” de microprocessador.

14. A imagem que se segue representa um evento fundamental num algoritmo de escalonamento “pre-emptive”. Qual?



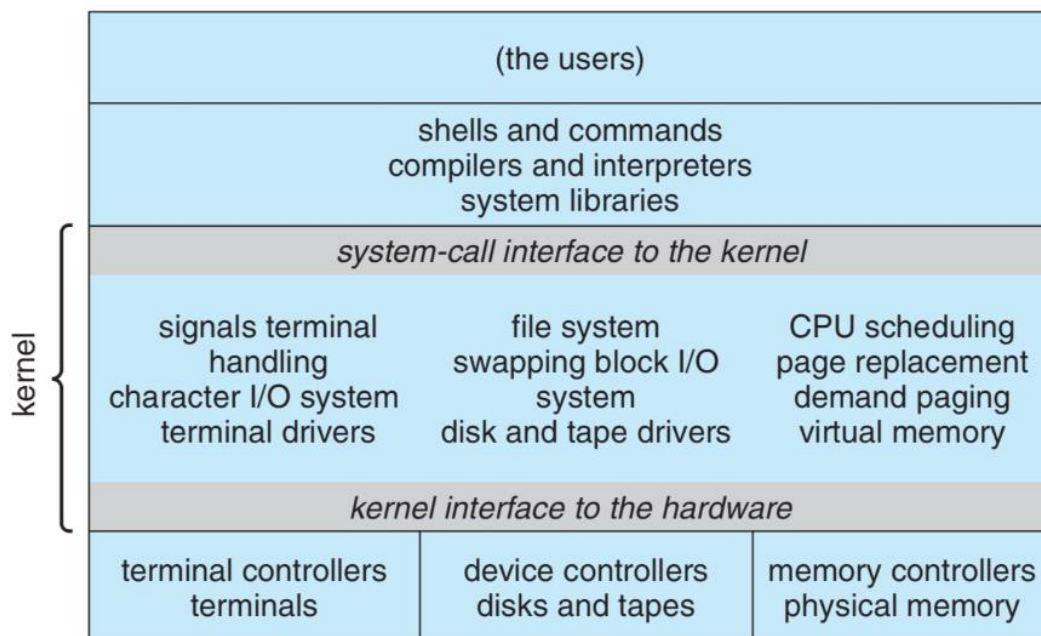
- um “backup” do sistema.
- uma operação de I/O.
- um “context switch”.

15. A “Memory Management Unit” (MMU) representada na figura seguinte não é adequada para sistemas operativos “multitasking” porque:



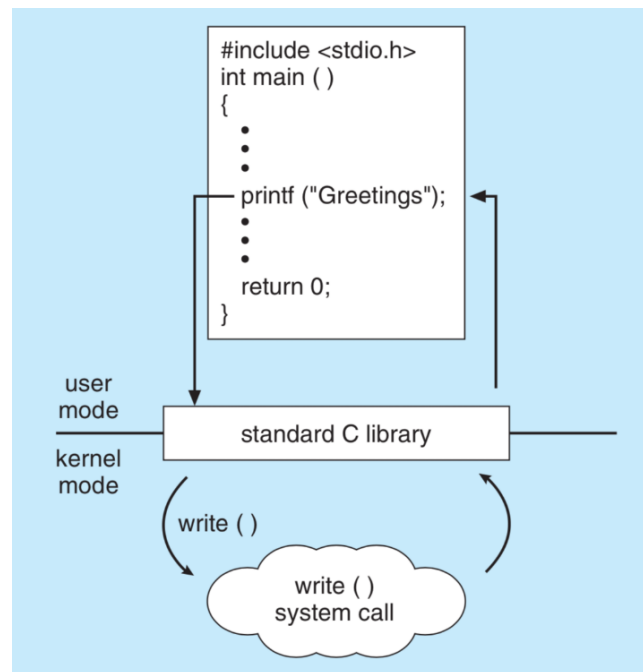
- o impacto da tradução dos endereços virtuais em endereços físicos no desempenho é enorme.
- implica espaços de endereçamento de processos colocados num único bloco de memória física.
- implica espaços de endereçamento de processos dispersos por vários blocos de memória física.

16. Na figura seguinte, que representa a estrutura de um sistema operativo UNIX, como se faz a interacção com o “kernel”?



- através de funções designadas por “system calls”.
- através de bibliotecas como a “Standard C Library”.
- um utilizador nunca interage directamente com o “kernel”.

17. A figura seguinte representa conceptualmente o funcionamento da chamada “printf” da “Standard C Library”. A relação entre o “printf” e a chamada ao sistema “write” é:



- cada invocação da função “printf” despoleta de imediato uma chamada ao sistema “write”.
- o “printf” usa buffers em memória para escrever temporariamente e quando enchem chama o “write”.
- o “printf” invoca a chamada ao sistema “write” apenas quando o processo corrente termina.

18. A implementação de algoritmos de escalonamento “pre-emptive” obrigou a inovações específicas no hardware e software, por exemplo:

- inibição de interrupções durante a execução de porções críticas de código.
- a utilização de espaços de endereçamento individuais para os processos.
- técnicas como o “Direct Memory Access” para transferir blocos grandes de dados.

19. O tempo médio de espera para a seguinte configuração de processos (P1-P2-P3-P4) na fila “ready”, com o algoritmo “Round-Robin” (RR) e um quantum de 5 é:

Process	Burst-Time
P1	21
P2	6
P3	11
P4	3

- 21.33
- 19.25
- 12.74

20. O tempo médio de espera na fila para a seguinte configuração de processos, com o algoritmo “Shortest Remaining Time First” (SRTF) é:

Process	Arrival-Time	Burst-Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

- a. 6.50
- b. 7.25
- c. 9.35

21. O “Completely Fair Scheduler” (CFS), utilizado no “kernel” do Linux organiza os processos:

- a. em filas simples;
- b. em filas múltiplas;
- c. numa árvore binária.

22. Tipicamente, os algoritmos “Earliest Deadline First” (EDF) e/ou “Rate Monotonic Scheduling” (RMS) são utilizados em sistema operativos que executam processos associados a tarefas:

- a. de tempo-real.
- b. interactivas.
- c. intensivas.

23. A diferença fundamental entre arquitecturas de 32 e 64 bits consiste no facto de as segundas permitirem:

- a. a execução de processos com espaços de endereçamento maiores.
- b. manter informação sobre mais processos na memória física.
- c. transferir processos completos para a memória física.

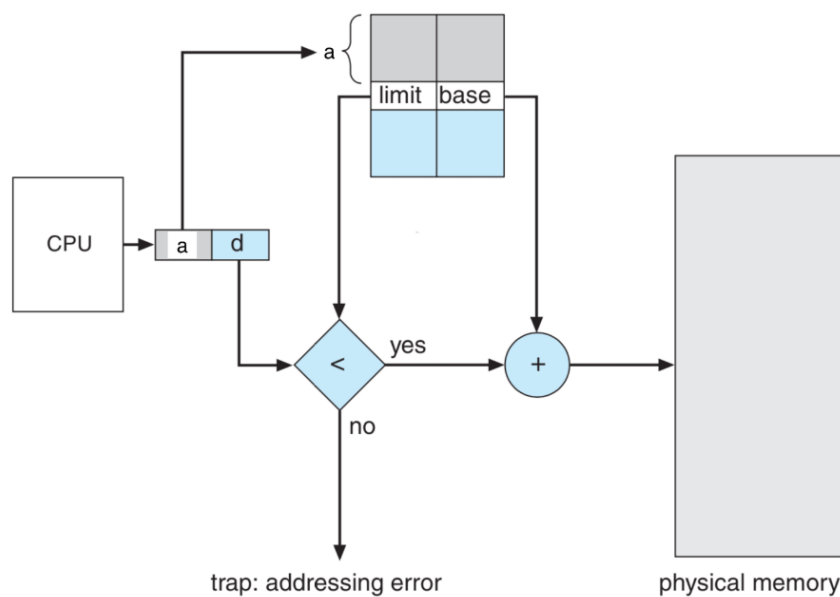
24. No CPU, a componente designada por “Memory Management Unit” (MMU) é responsável por:

- a. atribuir espaço aos processos que aguardam execução mantendo a integridade da memória física.
- b. garantir a integridade da memória e traduzir endereços virtuais em endereços na memória física.
- c. detectar acessos indevidos a zonas de memória e transferir controlo para o sistema operativo.

25. A priori, uma desvantagem das técnicas de segmentação e paginação relativamente a técnicas que mapeiam processos completos em zonas contíguas de memória consiste no facto da tradução de endereços virtuais em endereços físicos:

- não poder ser feita pelo hardware (MMU).
- envolver a consulta de tabelas em memória.
- as MMUs respectivas serem muito complexas.

26. A imagem que se segue representa uma MMU. O circuito suporta segmentação, paginação ou nenhuma das duas? Porquê?



- segmentação, porque a tabela de tradução tem o valor “limit” para cada entrada, indicando que os blocos de memória podem ter tamanhos diferentes.
- paginação, porque a tabela de tradução tem o valor “limit” para cada entrada, indicando que os limites das páginas estão a ser verificados.
- nenhuma, porque a existência da tabela com valores “limit” e “base” por entrada indica que esta guarda a posição de processos completos na memória.

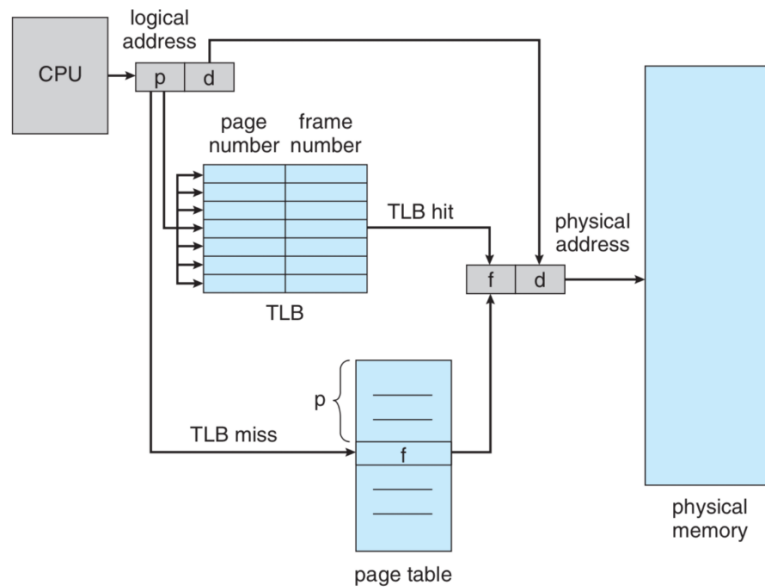
27. Uma vantagem significativa da técnica da paginação é:

- não provoca a fragmentação externa da memória física.
- todos os endereços traduzidos são válidos no programa.
- as páginas têm tamanho variável e estritamente necessário.

28. Na técnica da segmentação:

- não ocorre a fragmentação externa da memória física.
- os endereços traduzidos são sempre válidos no programa.
- a MMU é simples pois os segmentos são do mesmo tamanho.

29. A imagem que se segue representa uma MMU. Quantos acessos à memória estão envolvidos na tradução de um endereço de memória virtual quando: 1) o TLB tem um hit, 2) caso contrário:



- a. 1 e 2.
- b. 2 e 1.
- c. 2 e 2.

30. Numa arquitectura de 32 bits, um sistema operativo com páginas de 2 KBytes produz tabelas de páginas com quantas entradas?

- a. 2^{30} .
- b. 2^{21} .
- c. 2^{11} .

31. Uma vantagem/desvantagem, respectivamente, de usar páginas mais pequenas num sistema operativo, é:

- a. diminuir a fragmentação interna / aumentar o “swapping” de páginas.
- b. diminuir a fragmentação externa / diminuir a localidade de informação.
- c. aumentar a localidade de informação / aumentar a fragmentação externa.

32. O seguinte programa recebe o nome de dois ficheiros como argumentos e:

```
/* includes & defines */

int main(int argc, char* argv[]) {
    /* check arguments */

    FILE* fp1 = fopen(argv[1], "w");
    if(fp1 == NULL) { /* error action */ }

    FILE* fp2 = fopen(argv[2], "r");
    if(fp2 == NULL) { /* error action */ }

    int bytes_read;
    char buffer[BUFFER_SIZE];
    while( (bytes_read = fread(buffer, 1, BUFFER_SIZE, fp2)) != 0)
        fwrite(buffer, 1, bytes_read, fp1);

    fclose(fp2);
    fclose(fp1);

    return EXIT_SUCCESS;
}
```

- a. substitui o conteúdo do primeiro pelo do segundo.
- b. substitui o conteúdo do segundo pelo do primeiro.
- c. concatena o conteúdo do segundo com o do primeiro.

33. Como explica que o seguinte programa dê valores distintos para a variável “val” que está localizada no mesmo endereço?

```
/* includes & defines */
int main(int argc, char* argv[]) {
    pid_t pid;
    int val;

    pid = fork();

    if (pid == 0) {
        val = 0;
        printf("child: val = %d, at addr = %p\n", val, &val);
        return EXIT_SUCCESS;
    } else {
        val = 1;
        wait(NULL);
        printf("parent: val = %d, at addr = %p\n", val, &val);
        return EXIT_SUCCESS;
    }
}

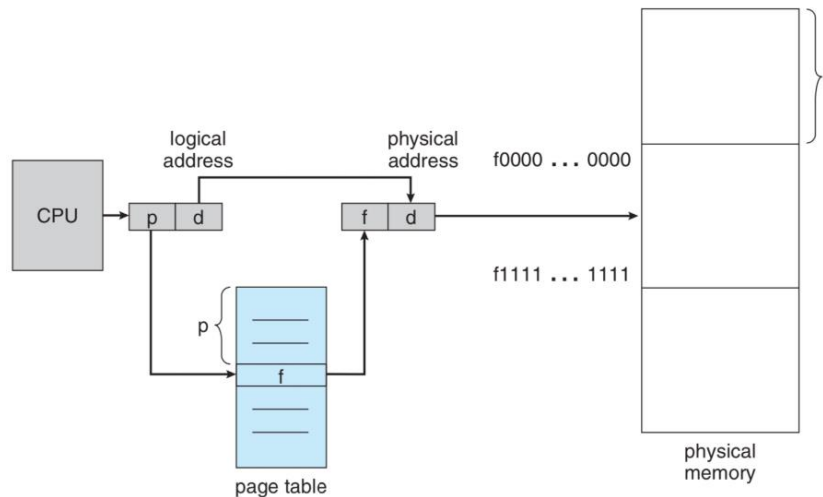
$ gcc -Wall val.c -o val
$ ./val
child: val = 0, at addr = 0x7ffeef275a38
parent: val = 1, at addr = 0x7ffeef275a38
```

- a. há um erro no programa, depois do “fork” os endereços têm de ser diferentes.
- b. “val” é alterada na memória física no mesmo endereço em instantes diferentes.
- c. os espaços de endereçamento dos processos são iguais e o endereço é virtual.

34. Se imprime o valor de um apontador obtendo, por exemplo, 0x7ffeef275a38, quantos bits tem a arquitectura em que o programa foi executado?

- a. 64 bits.
- b. 32 bits.
- c. 12 bits.

35. Na imagem da MMU que se segue, não é feita nenhuma verificação do “offset” do endereço para garantir que este cai sempre dentro da página porque:



- a. essa verificação é feita por software, pelo sistema operativo, e não pelo hardware da MMU.
- b. o hardware de tradução de endereços fica mais rápido mesmo que possa gerar erros ocasionais.
- c. a divisão em bits dos endereços virtuais garante que “offset” é inferior ao tamanho duma página.

36. A técnica da segmentação:

- a. preserva a localidade dos dados e das instruções de um programa.
- b. preserva a localidade dos dados mas não das instruções de um programa.
- c. em geral, não preserva a localidade dos dados nem das instruções.

37. Com as novas as arquitecturas de microprocessadores (e respectivas MMUs) de 64 bits:

- a. as novas MMUs apresentam o mesmo nível de suporte para a segmentação e para a paginação.
- b. o suporte para segmentação ao nível do hardware tem desaparecido e a paginação é preferida.
- c. o suporte para paginação ao nível do hardware tem desaparecido e a segmentação é preferida.